```
.686
.model flat
public _get_RPK
extern _mul_24@8

.code

Char_to_value PROC
    cmp al, 'B'
    jne check_C
    mov al,0
    ret
Check_C:
    cmp al, 'C'
    jne check_D
    mov al,1
    ret
Check_D:
    cmp al, 'D'
    jne check_F
    mov al,2
    ret
Check_F:
    cmp al,'F'
    jne check_G
    mov al,3
    ret
Check_G:
    cmp al,'G'
    jne check_H
    mov al,4
    ret
Check_H:
    cmp al,'H'
    jne check_J
    mov al,5
    ret
~~check_J~~
~~check_K~~
Check_J:
    cmp al, 'J'
    jne check_K
    mov al,6
    ret
Check_K:
    cmp al,'K'
    jne check_M
    mov al,7
    ret
Check_M:
    cmp al, 'M'
    jne check_P
    mov al,8
    ret
Check_P:
    cmp al,'P'
    jne check_Q
    mov al,9
    ret
Check_Q:
    cmp al,'Q'
    jne check_R
    mov al,10
    ret
Check_R:
    cmp al,'R'
    jne check_T
    mov al,11
    ret
Check_T:
    cmp al,'T'
    jne check_V
    mov al,12
    ret
Check_V:
    cmp al,'V'
    jne check_W
    mov al,13
    ret
Check_W:
    cmp al,'W'
    jne check_X
    mov al,14
    ret
Check_X:
    cmp al,'X'
    jne check_Y
    mov al,15
    ret
Check_Y:
    cmp al,'Y'
    jne check_2
    mov al,16
    ret
Check_2:
    cmp al,2
    jne check_3
    mov al,17
    ret
Check_3:
    cmp al,3
    jne check_4
    mov al,18
    ret
Check_4:
    cmp al,4
    jne check_6
    mov al,19
Check_6:
    cmp al,6
    jne check_7
    mov al,20
    ret
Check_7:
    cmp al,7
    jne check_8
    mov al,21
    ret
Check_8:
    cmp al,8
    jne check_9
    mov al,22
    ret
Check_9:
    cmp al,9
    jne invalid_char
    mov al,23
    ret
invalid_char:
    mov al,0
    ret
Char_to_value endp

~~_get_RPK PROC~~
_get_RPK PROC
    push ebp
    mov ebp,esp
    push ebx
    push esi
    push edi
    sub esp,15
    mov edi,esp
    xor eax,eax
    mov ecx,15
    rep stosb
    mov edi,esp
    mov esi,[ebp+8]
    add esi,6
    mov ecx,4
process_group:
    push ecx
    mov ecx,5
process_char:
    mov al,[esi]
    inc esi
    call char_to_value
    push esi
    push edi
    push edi
    push edi
    pop edi
    pop esi
    movzx eax,al
    add [edi],al
    mov ebx,1
carry_loop:
    adc byte ptr [edi+ebx]
    inc ebx
    cmp ebx,14
    jbe carry_loop
    loop process_char
    pop ecx
    cmp ecx,1
    je no_hyphen
    inc esi
no_hyphen:
    loop process_group
    mov eax,[edi+8]
    bswap eax
    add esp,15
    pop edi
    pop esi
    pop ebx
    pop ebp
    ret
_get_RPK endp
END
```